Introdução à Computação
endereços de memória e ponteiros

Alexandre Rademaker

memória I

Todo arquivo em um computador é armazenado no **disk driver**, seja um (hard disk driver, mecânico) HDD ou (solid-state drive) SDD.

Mas os **disk drivers** são apenas lugares para armazenamento, os dados são manipulados por pogramas na RAM. Então quando um programa é carregado e precisa manipular dados, ele lê os dados do disco para a RAM.

A memória RAM é basicamente um **enorme array de bytes** (8 bits) somando 512BM, 1GB, 4GB, 16GB, 32GB etc.

memória II

Lembrando que cada tipo de dados demanda uma quantidade de bytes para armazenamento de valores daquele tipo.

Data Type	Size (in bytes)
int	4
char	1
float	4
double	8
long long	8
string	???

memória III

Usamos arrays para armazenar dados de forma contínua e em unidades de tamanho uniformes. O acesso aleatório a qualquer posição é então eficiente.

Apenas indicando a posição sequencial no array e multiplicando-a pelo tamanho dos elementos, temos o endereço do valor desejado.

O mesmo ocorre com qualquer posição da memória, que também tem um endereço associado.

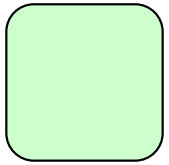
memória IV

		H		65				H	i	\0	
0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB

```
char c = 'H';
int limit = 65;
char g[] = "Hi";
```

ponteiros são endereços!

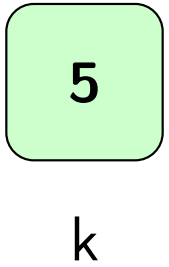
usando ponteiros I



k

```
int k;
```

usando ponteiros II

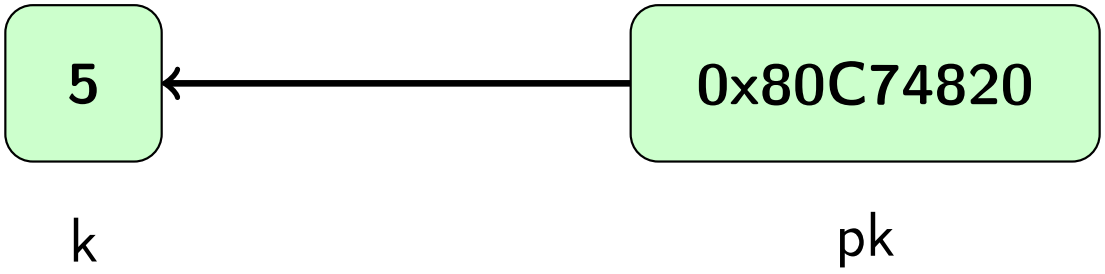



```
int k;
k = 5;
```

usando ponteiros III

```
int k;
k = 5;
int *pk;
```

usando ponteiros IV




```
int k;
k = 5;
int *pk;
pk = &k;
```

usando ponteiros V

Um ponteiro é apenas uma variável cujo valor é um endereço de memória. O tipo descreve o dado localizado no endereço.

O ponteiro mais simples de C é o NULL. O NULL aponta para nenhum lugar, o que será bastante útil. O elemento neutro.

Ao criar um ponteiro, se um valor não for atribuído imediatamente (um endereço), é adequado sempre atribuir o valor NULL.

Você pode verificar se o ponteiro é NULL com o operador de igualdade (`==`).

usando ponteiros VI

A forma mais fácil de obter um ponteiro é extrair o endereço de uma variável já existente, para isso usamos o operador `&`.

Se `x` for uma variável do tipo `int`, então `&x` é o endereço de `x`. E podemos criar um ponteiro para `x` com valor igual a `&x`.

Se `arr` é um array de doubles, então `&arr[i]` é um ponteiro para double cujo valor é o endereço do iéssimo elemento de `arr`.

O nome de um array é basicamente um ponteiro para o seu primeiro elemento.

usando ponteiros VII

Com ponteiros, temos uma forma altenativa de passar dados para funções.

Até agora, valores eram passados **por valor**, o que significa que uma cópia do valor armazenado na variável referenciada na chamada da função era passada.

A única excessão até agora ocorria com arrays.

usando ponteiros VIII

Usando ponteiros, podemos passar a variável sendo referenciada na chamada, ou melhor, uma referência à ela, seu endereço.

Isto significa que uma mudança feita na variável no corpo da função chamada irá mudar o valor da variável mesmo quando a função chamada retornar.

Isto pode ser bom ou ruim! Teremos que ter mais cuidado.

usando ponteiros IX

Para inspecionar ou modificar um determinado endereço da memória apontada por um ponteiro precisamos **dereferenciando** o ponteiro.

Se tivermos um ponteiro chamado `pc`, então `*pc` é o valor que está armazenado no endereço da memória armazenado na variável `pc`.

usando ponteiros X

Então existem dois contextos de uso do operador `*`. Como **dereferenciador**, ele 'vai para a referência' e acessa o dado que estiver naquela localização da memória.

Se tentarmos **dereferenciar** um ponteiro cujo valor é `NULL`, teremos um

Segmentation fault

Isto, na verdade, é um comportamento esperado. É uma defesa contra um acesso indesejado a um ponteiro desconhecido.

usando ponteiros XI

Mais uma coisa irritante sobre o operador `*`. Ele é parte tanto do tipo quanto do nome de uma variável.

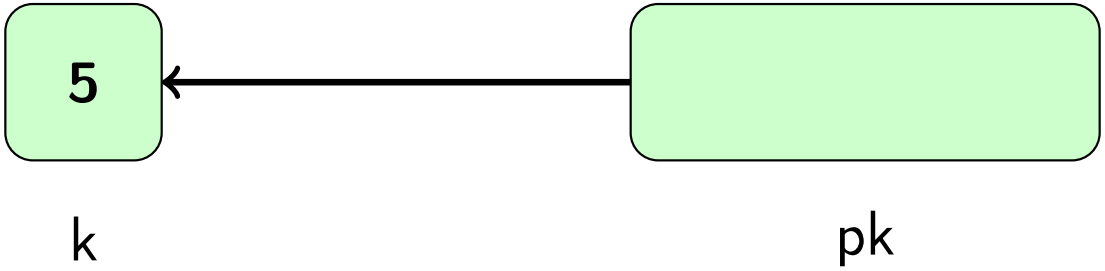
src/decl.c

```
1  #include <stdio.h>
2
3  int main(void) {
4    int* px, py, pz;
5    int *pa, *pb, *pc;
6
7    printf("px %lu py %lu pz %lu \n",
8     sizeof(px), sizeof(py), sizeof(pz));
9    printf("pa %lu pb %lu pc %lu \n",
10    sizeof(pa), sizeof(pb), sizeof(pc));
11 }
```

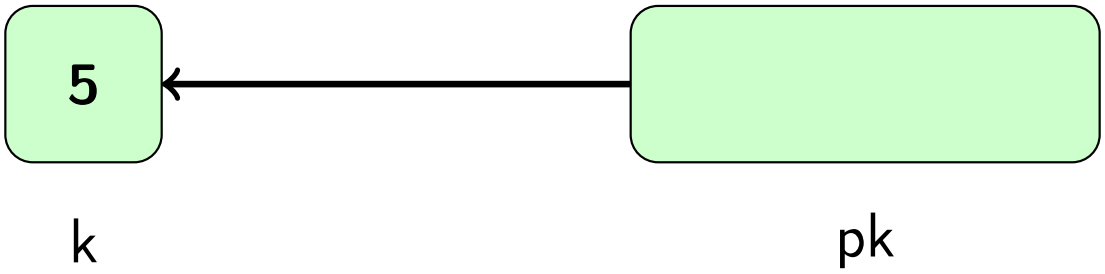
usando ponteiros XII

Data Type	Size (in bytes)
int	4
char	1
float	4
double	8
long long	8
char*, int*, double*, float* ...	8

passo a passo I

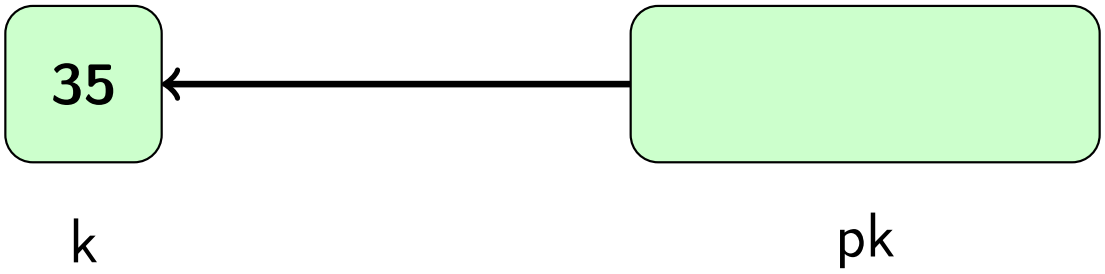


passo a passo II



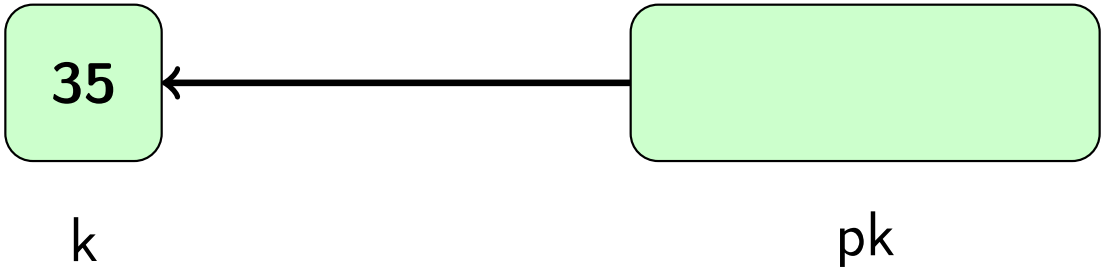

```
*pk = 35;
```

passo a passo III



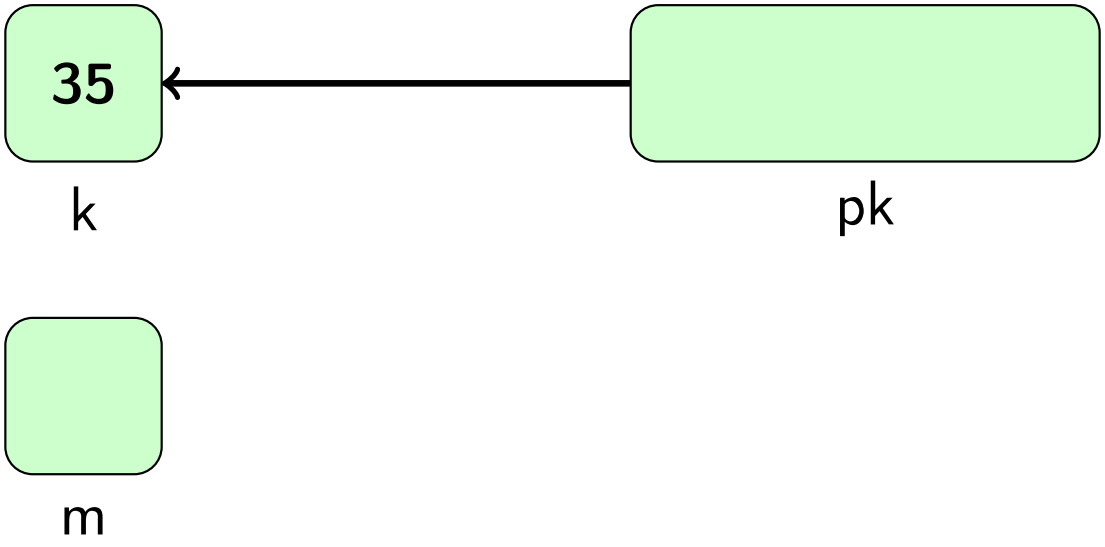

```
*pk = 35;
```

passo a passo IV



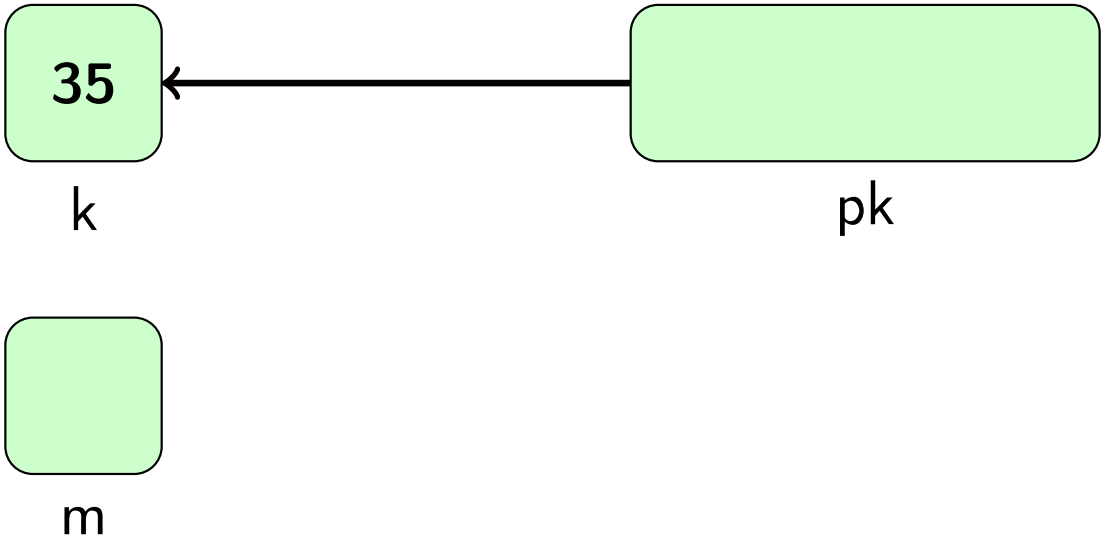

```
*pk = 35;
int m;
```

passo a passo V



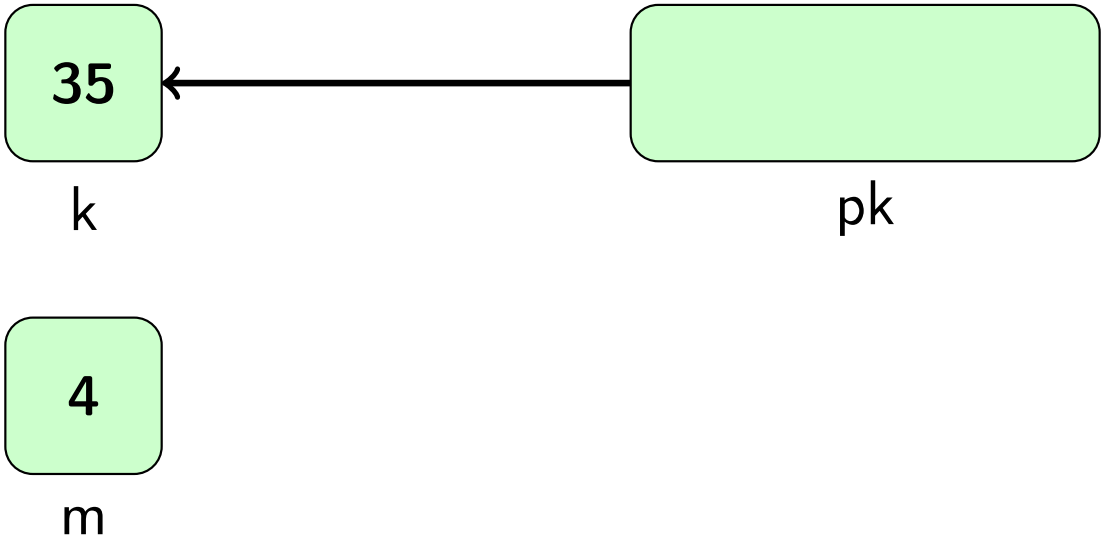

```
*pk = 35;
int m;
```

passo a passo VI



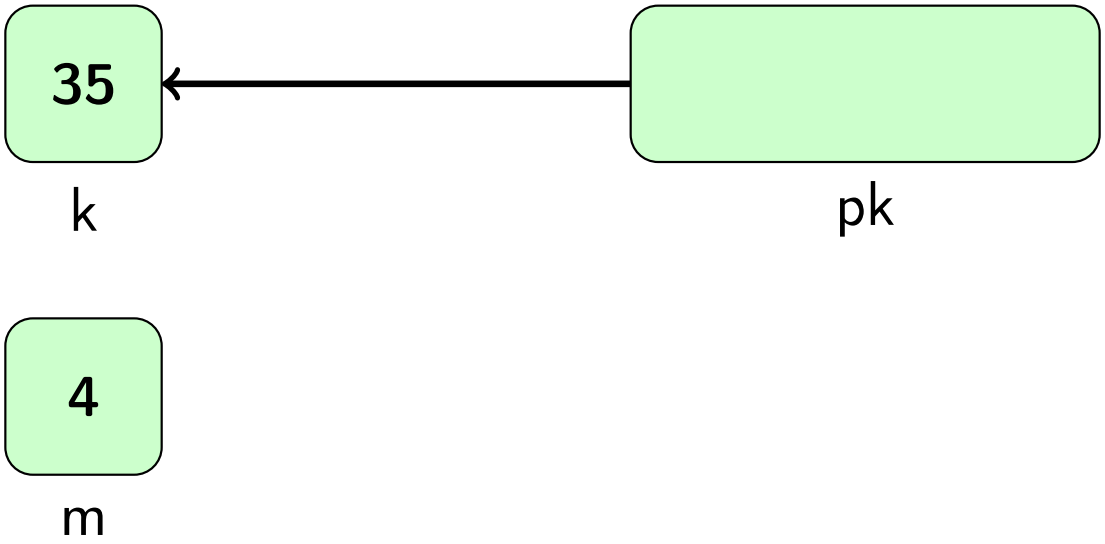

```
*pk = 35;
int m;
m = 4;
```

passo a passo VII



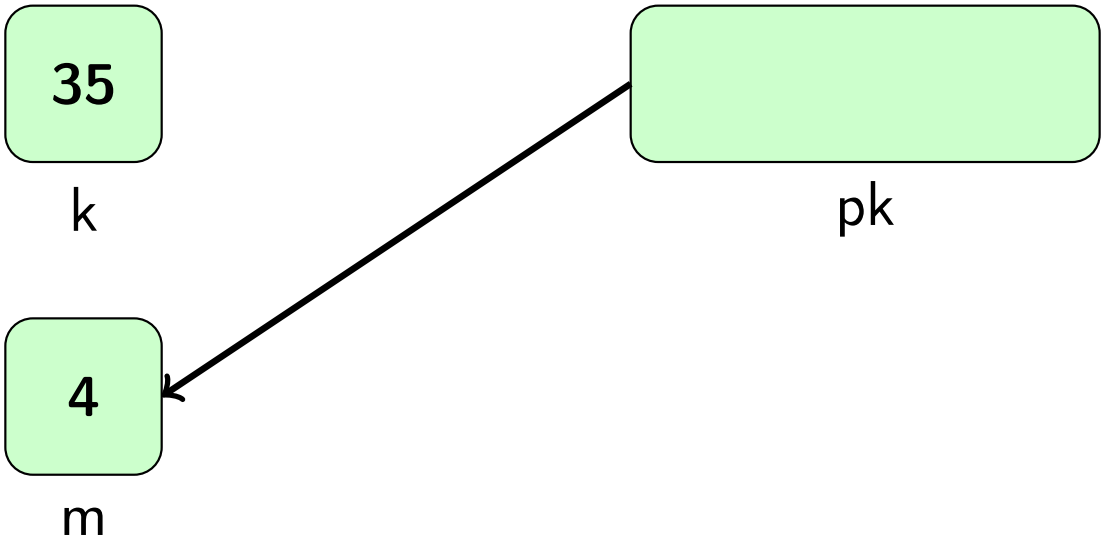

```
*pk = 35;
int m;
m = 4;
```

passo a passo VIII




```
*pk = 35;
int m;
m = 4;
pk = &m;
```

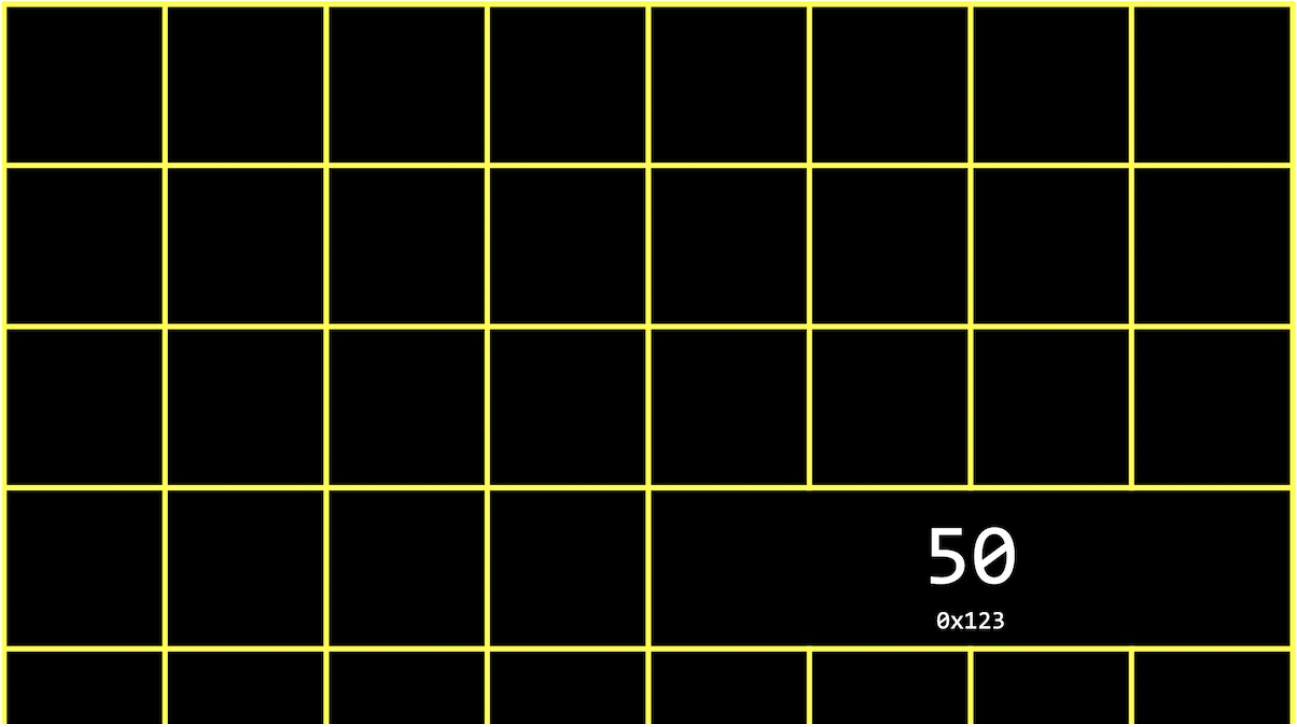
passo a passo IX




```
*pk = 35;
int m;
m = 4;
pk = &m;
```

exemplos I

O que fizemos foi preencher uma dada posição de memória, que tem um endereço associado, com o valor 50.



exemplos II

src/address01.c

```c
#include <stdio.h>

int main(void) {

  int n = 50;
  int *p = &n;

  printf("at %p the value is %d\n", p, *p);
  printf("at %p the value is %d\n", &n, n);
}
```

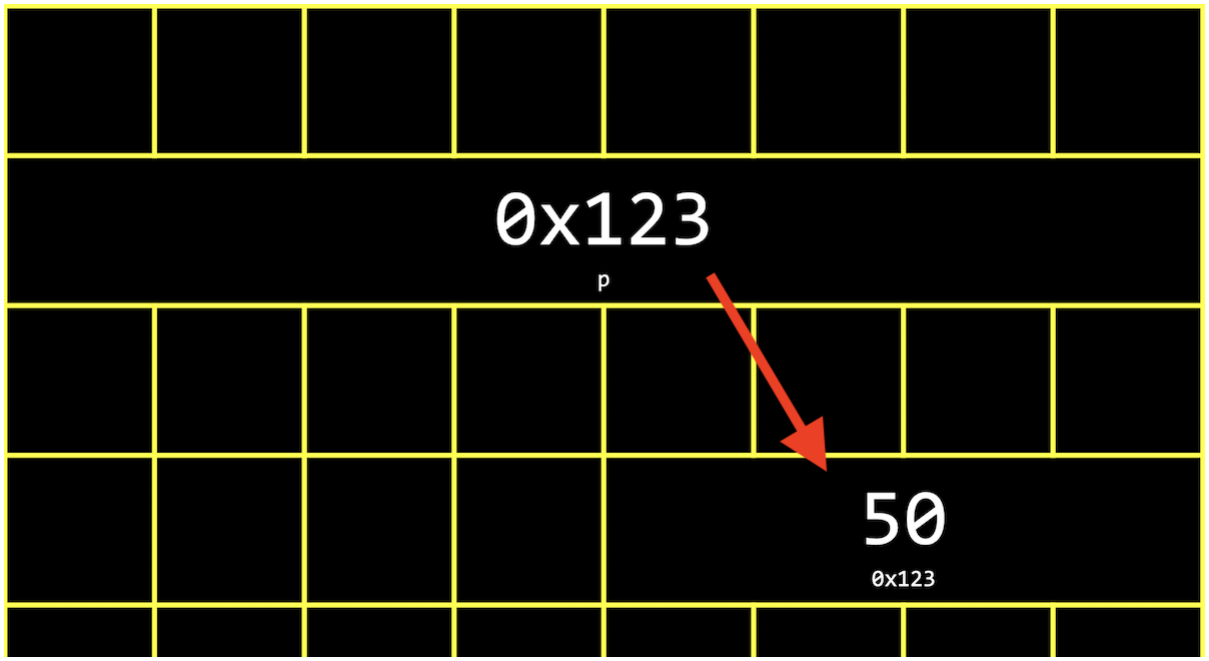
`int *p` ou `int* p` a variável `p` aponta para um endereço onde um valor inteiro está armazenado

`&n` o endereço da variável `n`

`*p` (**dereference operator**) retorna o valor armazenado no endereço armazenado em `p`

exemplos III

Como declaramos `p` como sendo do tipo `int *`, o compilador saberá que `*p` é um inteiro, ou seja, o número de bytes que devem ser lidos.



Note que `p` também tem um endereço! Observe que `p` ocupa 8 bytes (64 bits).